



OPENBOOK · INTEGRATIONSSEMINAR

ELISA

Technische Übergabe- und Abschlussdokumentation für die nächste Projektgruppe

Stand: 04.07.2026

Inhalt

Zweck dieses Dokuments	3
Gesamtzustand.....	3
Kursmaterial-Synchronisation und Chat-Anfragefluss	4
Zentrale Rolle des Orchestrators.....	4
Technischer Gesamtaufbau.....	5
Assistant, RAG und Orchestrator.....	6
Learning-App	7
Gamification	7
Content-Integration.....	8
Student Dashboard.....	8
Teacher-Frontend.....	9
Wichtige Abläufe	9
Kursvorbereitung durch Lehrende	9
Lernen durch Studierende.....	9
Quiz- und Exam-Flow.....	10
Technische Zusatzdokumentation.....	10
Bekannte Punkte für die Übergabe	11
Performance prüfen	11
Modellwahl prüfen	11
Memory und Spiele schärfen	11
Sokratischen Hilfemodus ergänzen	11
Quiz- und Exam-Regeln entscheiden.....	11
Gamification didaktisch finalisieren	11
Lernstandsmodell erweitern	11
Teacher-Workflow stabilisieren	12
Testdaten und Demo-Stand vorbereiten	12
Abhängigkeiten und lokale Robustheit prüfen.....	12
Alte Kommentare und technische Notizen bereinigen	12
Empfohlene erste Schritte für die nächste Gruppe.....	12
Kurzfasit.....	12

Zweck dieses Dokuments

Dieses Dokument zeigt, wo ELISA im OpenBook-Projekt aktuell steht. Es richtet sich an die nächste Projektgruppe und an Personen, die nachvollziehen müssen, welche Bausteine umgesetzt wurden und wo sie im Quellcode liegen.

Die Dokumentation ersetzt kein OpenBook-Setup-Handbuch. Allgemeine Installation, Poetry, Django, Admin und OpenBook-Grundfunktionen bleiben in der bestehenden OpenBook-Dokumentation. Hier geht es um ELISA: Assistant/RAG, Learning, Gamification, Student Dashboard, Teacher-Frontend, Quiz, Exam, Games und die Übergabepunkte für die nächste Gruppe.

So wird dieses Dokument genutzt:

1. Zuerst den Gesamtzustand lesen, um das fachliche Bild zu verstehen.
2. Danach den passenden Funktionsbereich öffnen, z. B. Assistant oder Teacher-Frontend.
3. Über die Artefakt-Tabellen die zuständigen Dateien finden.
4. Für Detailarbeit die verlinkten technischen Zusatzdokumente und den Code lesen.
5. Am Ende die Handover-Hinweise als Startpunkt für die nächste Iteration verwenden.

Gesamtzustand

ELISA existiert aktuell als breiter Proof of Concept in OpenBook. Der Prototyp verbindet kursbezogene KI-Unterstützung, Lernstandsverfolgung, Quiz- und Exam-Übungen, Gamification sowie getrennte Oberflächen für Studierende und Lehrende.

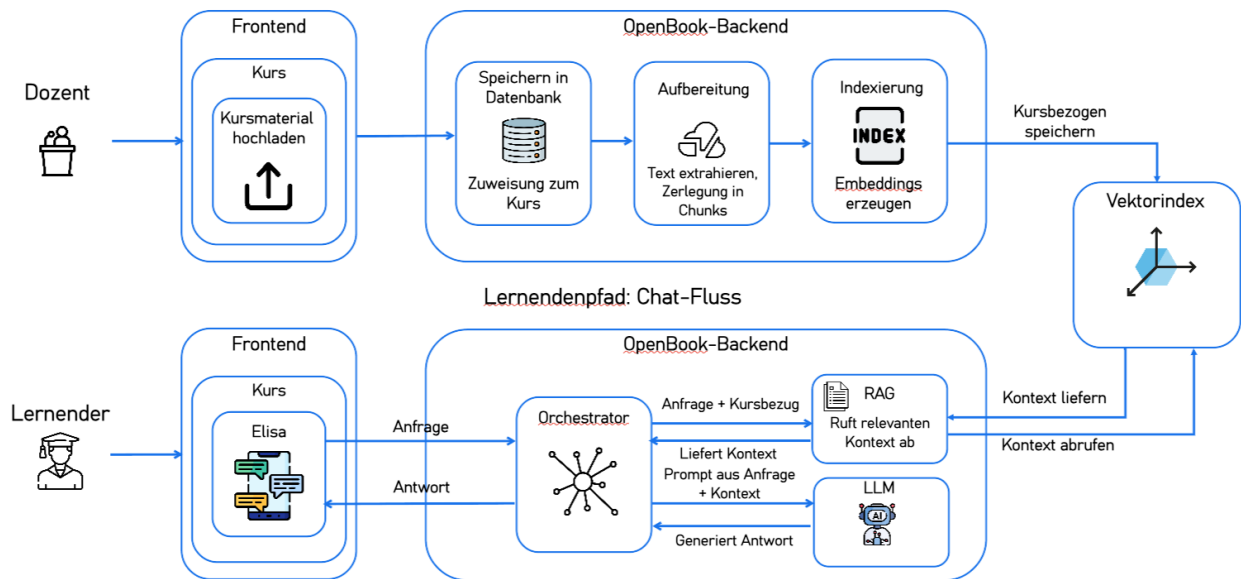
Fachlich läuft der Ablauf in sechs Schritten ab:

1. Lehrende legen im Teacher-Frontend Kurse, Textbooks und Seiten an.
2. Inhalte werden als Assistant-Dokumente synchronisiert und für RAG indexiert.
3. Studierende öffnen den Kurs im Dashboard.
4. Der Kurs-Chat, Quiz und Exam nutzen Kursmaterial und Lernstand als Kontext.
5. Lernaktivitäten aktualisieren LearningState, QuizResult und Gamification.
6. Punkte, Level, Streaks und Skills werden im Dashboard sichtbar.

Die Umsetzung ist noch kein fertiges Produkt. Die Hauptbausteine sind vorhanden und integriert, aber die nächste Gruppe sollte Stabilität, Performance, Modellwahl, didaktische Regeln und Testdaten gezielt nachziehen.

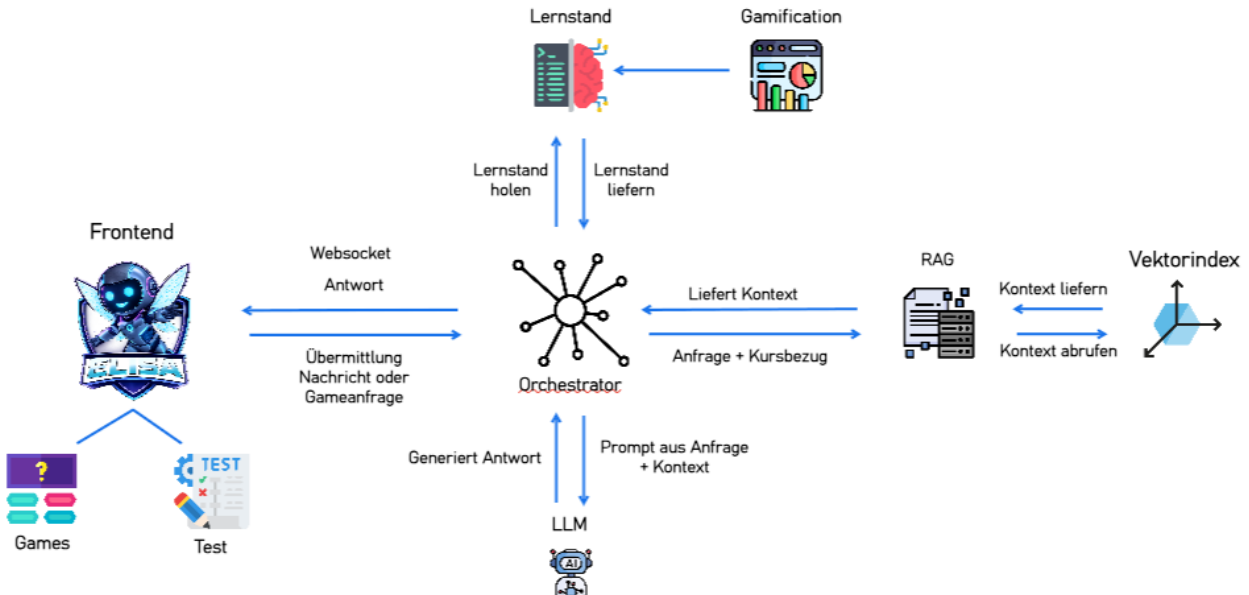
Die folgenden beiden Abbildungen veranschaulichen diesen Ablauf technisch: zunächst die Synchronisation von Kursmaterial und der Chat-Anfragefluss, anschließend die zentrale Rolle des Orchestrators.

Kursmaterial-Synchronisation und Chat-Anfragefluss



Wenn eine Lehrperson Kursmaterial hochlädt, speichert und indiziert das Backend es automatisch im Vektorindex. Stellt später ein Studierender im Chat eine Frage, greift ELISA auf genau diesen Index zurück und beantwortet sie mit Unterstützung des LLM.

Zentrale Rolle des Orchestrators



Der Orchestrator sitzt zwischen Frontend und den Fachservices. Er kennt den aktuellen Lernstand, holt bei Bedarf über RAG passenden Kontext aus dem Vektorindex und gibt Anfrage und Kontext an das LLM weiter. Außerdem meldet er Aktivitäten an die Gamification. Die fertige Antwort geht zurück ans Frontend, das unter anderem auch Games und Testfunktionen einbindet.

Umgesetzte ELISA-Bausteine

Bereich	Aktueller Stand	Übergabewert
Assistant/RAG	Kursbewusster Assistant mit Orchestrator, RAG-Kontext, Prompt Builder, Quiz-/Exam-Services und Mistral-Client.	Grundlage für materialbezogene KI-Antworten und Aufgaben.
Learning	LearningState, QuizResult, Learning-API, Kontextservice und Brücke zu Gamification.	Speichert Lernposition, abgeschlossene Seiten und Quiz-Ergebnisse.
Gamification	Punkte, Level, Streaks, Rewards, CourseProgress, Skill-Katalog und SkillProgress.	Macht Lernaktivität sichtbar und belohnt Übungen.
Dashboard	Svelte-5-Frontend für Kurse, Chat, Skript, Quiz, Exam, Games, Skills und Profil.	Zentrale Studierendenoberfläche.
Teacher-Frontend	Svelte-5-Frontend für Kursliste, Kursdetails, Content-Editor, Skill-Tagging und Einschreibung.	Zentrale Lehrendenoberfläche für ELISA-Kursarbeit.
Games	Memory, Flashcards und Hangman als Lernspiele aus Kursinhalten.	Zeigt spielerische Lernformate, braucht aber fachliche Schärfung.

Technischer Gesamtaufbau

ELISA erweitert bestehende OpenBook-Modelle, statt eine komplett getrennte Plattform zu sein. Kurse, Textbooks, Textbook-Seiten, Rollen und Berechtigungen kommen aus OpenBook. ELISA ergänzt darauf eigene Apps und Frontends.

Die Backend-Apps liegen unter `src/openbook/assistant/`, `src/openbook/learning/` und `src/openbook/gamification/`. Die neuen Microfrontends liegen unter `src/frontend/dashboard/` und `src/frontend/teacher/`. Zusätzlich wurden bestehende Content-ViewSets erweitert, damit Textbook-Seiten automatisch in Assistant-Dokumente synchronisiert werden.

Kursmaterial wird dabei in mehreren Schritten synchronisiert:

1. Eine Lehrperson erstellt oder importiert Textbook-Seiten.
2. `TextbookDocumentSyncService` rendert daraus eine Markdown-Datei.
3. Diese Datei wird als `AssistantDocument` gespeichert.
4. Der RAG-Client erzeugt Chunks und Embeddings.
5. Chat, Quiz und Exam können den Kurskontext abrufen.

Assistant, RAG und Orchestrator

Alle KI-Funktionen liegen im Assistant-Bereich, gesteuert vom AssistantOrchestrator. Viel Fachlogik steckt darin nicht. Er liest die Anfrage, prüft die Berechtigung und reicht sie an die passenden Services weiter, etwa für Chat, Quiz, Exam, Learning oder Kontext.

Assistant-Artefakte

Artefakt	Aufgabe
assistant/services/orchestrator.py	Einstiegspunkt für Chat, Seitenereignisse, Quiz, Exam und Grading. Verbindet Request-Kontext, Learning und KI-Services.
assistant/services/llm_client.py	Kapselt den Mistral-Client, das Chatmodell <code>mistral-small-latest</code> und das Embedding-Modell <code>mistral-embed</code> .
assistant/services/rag_client.py	Indexiert Assistant-Dokumente, sucht relevante Chunks über <code>sqlite-vec</code> und baut RAG-Kontext für Antworten.
assistant/services/vector_index.py	Verwaltet die technische <code>sqlite-vec</code> -Tabelle für Embedding-Suche und Aufräumen gelöschter Dokumente.
assistant/services/textbook_sync.py	Rendert Textbook-Seiten in Markdown, schreibt daraus Assistant-Dokumente und startet den Index-Neuaufbau.
assistant/services/request_context.py	Löst Course- und Page-IDs auf und prüft, ob der Nutzer den Kurs verwenden darf.
assistant/services/course_context.py	Baut fallbackfähigen Kurs- und Textbook-Kontext aus Kursdaten, Seiten und Skills.
assistant/services/prompt_builder.py	Baut die Prompts für Kursfragen, Quiz-Generierung, Exam-Generierung und Freitextbewertung.
assistant/services/chat.py	Führt kursbezogene Chat-Anfragen über den LLM/RAG-Pfad aus und meldet Chat-Aktivität an Learning/Gamification.
assistant/services/quiz_service.py	Erzeugt Multiple-Choice-Quizze aus RAG- oder Kurskontext und bewertet eingereichte Antworten serverseitig.
assistant/services/quiz_generation.py	Enthält Datenklassen, Serialisierung und Parser für generierte Quizfragen.
assistant/services/exam_service.py	Erzeugt und bewertet gemischte Exams mit Multiple-Choice- und Freitextfragen.
assistant/services/exam_generation.py	Enthält Datenklassen, Serialisierung, Punktebegrenzung und Parser für Exams.

Die wichtigsten Modelle liegen in `src/openbook/assistant/models/`. `AssistantDocument` und `AssistantDocumentChunk` speichern RAG-Dokumente und Chunks. `ChatSession` und `ChatMessage` speichern Chatverläufe. `QuizAttempt` und `ExamAttempt` halten generierte Aufgaben und Ergebnisse serverseitig fest.

Die WebSocket-Anbindung läuft über `src/openbook/ai/consumers/chat.py`. Das aktuelle Dashboard verwendet kursbezogene Kanäle wie `/ai/courses/<course-id>/chat`. Wichtig: `camelize = False` muss erhalten bleiben, weil das Frontend `snake_case`-Felder wie `page_id` und `points_awarded` erwartet.

Learning-App

Die Learning-App speichert Lernzustände und stellt den Lernkontext für Assistant und Gamification bereit. Sie verbindet Kursinhalt, Studierendenaktivität und Belohnungssystem miteinander.

Learning-Artefakte

Artefakt	Aufgabe
learning/models/state.py	LearningState speichert pro Nutzer und Kurs letzte Seite, abgeschlossene Seiten, Kursabschluss und letzten Zugriff.
learning/models/quiz_result.py	QuizResult speichert den letzten Quizstand pro Nutzer und Seite.
learning/services/context.py	Lädt Lernstand, Quiz-Ergebnisse und Kursfortschritt und rendert daraus promptfähigen Lernkontext.
learning/services/activity.py	Zentraler Service für Lernereignisse: Chat-Frage, Seite geöffnet, Seite abgeschlossen, Quiz- und Exam-Ergebnis.
learning/services/rewards.py	Kapselt die Übergabe von Learning-Ereignissen an Gamification-Services.
learning/viewsets/state.py	REST-Endpunkte für record-page-opened, mark-page-completed und complete-course.
learning/signals.py	Meldet Lernaktivität an den Streak-Mechanismus.

Das Dashboard verwendet die REST-Endpunkte aus `learning/states/` für Seitenaufrufe, Seitenabschluss und Kursabschluss. Quiz und Exam laufen dagegen über den Course-Chat-WebSocket, weil dort Aufgaben erzeugt, bewertet und belohnt werden.

Gamification

Gamification macht Lernaktivität sichtbar, vergibt aber keine Noten. Stattdessen sorgt die App für Motivation und Feedback. Punkte werden global und pro Kurs geführt. Skills haben zusätzlich eigene Level und Fortschrittswerte.

Gamification-Artefakte

Artefakt	Aufgabe
gamification/models/account_progress.py	Globaler Punktestand und globales Level pro Account.
gamification/models/course_progress.py	Kurspunkte, Kurslevel und Kursfortschritt pro Account und Kurs.
gamification/models/level_threshold.py	Gemeinsame Level-Schwellen für globale und kursbezogene Fortschritte.
gamification/models/account_streak.py	Aktueller und längster Daily Streak.
gamification/models/account_activity_day.py	Tagesweise Aktivitätsprotokolle für Streaks.
gamification/models/skill.py	Globaler Skill-Katalog.
gamification/models/skill_progress.py	Nutzerspezifischer Fortschritt pro Skill.
gamification/models/reward.py	Konfigurierbare Belohnungen.
gamification/models/reward_event_log.py	Ereignislog für Punkte und Belohnungen.
gamification/services/course.py	Vergibt Kurspunkte, aktualisiert CourseProgress und schreibt Reward-Events.
gamification/services/learning_rewards.py	Vergibt Quiz-, Exam- und Chat-Belohnungen mit Anti-Farming-Regeln.
gamification/services/skill.py	Erhöht SkillProgress und berechnet Skill-Level.
gamification/services/streak.py	Pflegt Daily Streaks und Aktivitätstage.
gamification/signals.py	Aktualisiert AccountProgress und Streaks nach RewardEventLog-Ereignissen.

Der zentrale Schreibpfad läuft über Services und RewardEventLog. Dadurch bleiben Punkte, Level und Streaks nachvollziehbar. Quiz- und Exam-Punkte werden verbesserungsbasiert vergeben: Nur ein besseres Ergebnis als bisher erzeugt neue Punkte.

Content-Integration

ELISA nutzt bestehende OpenBook-Content-Modelle. Die relevanten Erweiterungen liegen bei Kursen, Textbooks und Textbook-Seiten.

Content-Artefakte

Artefakt	Aufgabe
content/viewsets/course.py	Enthält <code>upload_textbook</code> . Lehrende können <code>.md</code> , <code>.markdown</code> , <code>.html</code> , <code>.htm</code> , <code>.txt</code> und <code>.pdf</code> hochladen. Daraus entstehen Textbook-Seiten und ein Assistant-Dokument.
content/viewsets/textbook.py	Synchronisiert Assistant-Dokumente, wenn Textbook-Metadaten geändert werden.
content/viewsets/textbook_page.py	Synchronisiert Assistant-Dokumente nach Erstellen, Ändern oder Löschen von Seiten.
content/services/chapter_extraction.py	Erkennt Kapitel aus hochgeladenem Text oder PDF und liefert Seiteninhalte für den Import.

Diese Integration ist wichtig, weil der Assistant nicht losgelöst von Kursmaterial arbeitet. Eine gespeicherte Seite kann später im Dashboard gelesen, als Assistant-Dokument heruntergeladen und als Kontext für Chat, Quiz, Exam oder Spiele verwendet werden.

Student Dashboard

Das Dashboard ist ein eigenständiges Svelte-5-Microfrontend unter `src/frontend/dashboard/`. Es wird serverseitig unter `/dashboard/` ausgeliefert. Die internen Seiten verwenden Hash-Routen.

Dashboard-Artefakte

Artefakt	Aufgabe
dashboard/src/index.ts	Einstiegspunkt, initialisiert Theme und mountet DashboardApp.
dashboard/src/components/DashboardApp.svelte	Shell mit Header, Router, Footer und ChatWidget.
dashboard/src/components/routes.ts	Hash-Routen: <code>/</code> , <code>/profile</code> , <code>/chat/:id</code> , <code>/quiz/:id</code> , <code>/exam/:id</code> , <code>/content/:id</code> und Games-Routen.
dashboard/src/stores/dashboard.store.ts	Lädt User, Fortschritt, Punkte, Skills, Kurse und Leaderboard.
dashboard/src/stores/ai-chat.store.ts	Kapselt Kurs-Chat, gespeicherte Sessions und WebSocket-Nachrichten.
dashboard/src/stores/quiz.store.ts	Erzeugt Quizze, reicht Antworten ein und verarbeitet Punkte-/Skill-Feedback.
dashboard/src/stores/exam.store.ts	Erzeugt, bewertet und lädt Exam-Versuche.
dashboard/src/api/learning.ts	Meldet Seitenaufrufe, Seitenabschluss und Kursabschluss an die Learning-API.
dashboard/src/data/course-content.ts	Baut die lesbare Kursinhaltsansicht aus Materialien, Seiten und Downloads.
dashboard/src/data/course-terms.ts	Extrahiert Begriffe und Flashcards aus Kursinhalten für Spiele.
dashboard/.../pages/ContentPage.svelte	Skriptansicht mit Seitenliste, Lesefortschritt und Downloads.
dashboard/.../pages/CourseChatPage.svelte	Vollbild-Kurs-Chat.
dashboard/.../pages/QuizPage.svelte	Textbook-Auswahl, Quiz-Generierung, Bearbeitung und Ergebnisanzeige.
dashboard/.../pages/ExamPage.svelte	Textbook-Auswahl, Exam-Generierung, Abgabe, Ergebnis und Historie.
dashboard/.../pages/GamesPage.svelte	Einstieg in Memory, Flashcards und Hangman.

Die Dashboard-Navigation ist für Studierende gedacht. Technische Details wie API-Pfade oder WebSocket-Actions gehören nicht ins Benutzerhandbuch, sind aber für die Weiterentwicklung in den Stores und API-Dateien nachvollziehbar.

Teacher-Frontend

Das Teacher-Frontend ist ein eigenständiges Svelte-5-Microfrontend unter `src/frontend/teacher/`. Es wird serverseitig unter `/teacher/` ausgeliefert. Der Einstieg ist für Mitglieder der Django-Gruppe Teacher geschützt.

Teacher-Artefakte

Artefakt	Aufgabe
<code>teacher/src/index.ts</code>	Einstiegspunkt, initialisiert Theme und mountet TeacherApp.
<code>teacher/src/components/TeacherApp.svelte</code>	Shell mit Header, Router und Footer.
<code>teacher/src/components/routes.ts</code>	Hash-Routen für Kursliste und Kursdetailseite.
<code>teacher/src/stores/teacher.store.ts</code>	Lädt aktuellen Lehrenden und dessen Kurse.
<code>teacher/src/api/courses.ts</code>	Kurse, Library Groups, Current User, Slug-Erzeugung.
<code>teacher/src/api/content.ts</code>	Textbooks, Seiten, Upload, Materialien, Page Ranges und Skills.
<code>teacher/src/api/enrollment.ts</code>	Usersuche, Student-Rolle und Kurseinschreibung.
<code>teacher/src/data/teacher.ts</code>	Wandelt API-Daten in View-Modelle für Kursliste und Content-Editor.
<code>teacher/.../pages/CourseListPage.svelte</code>	Kurskarten, New course-Dialog und Kurslöschung.
<code>teacher/.../pages/CourseDetailPage.svelte</code>	Kursdetailseite mit Overview, Students und Content.
<code>teacher/.../panels/OverviewPanel.svelte</code>	Bearbeitung von Kursdetails.
<code>teacher/.../panels/StudentsPanel.svelte</code>	Einschreiben und Ausschreiben von Studierenden.
<code>teacher/.../panels/ContentPanel.svelte</code>	Textbook-Anlage, Upload, Seiteneditor, Vorschau, Import und Skill-Tagging.

Im Lehrendenworkflow dreht sich das meiste um die Content-Pflege. Ein gespeichertes Textbook oder eine gespeicherte Seite stößt serverseitig die Assistant-Synchronisation an. Das ist die Grundlage dafür, dass Studierende im Dashboard kursbezogene Inhalte sehen und ELISA sinnvollen Kontext erhält.

Wichtige Abläufe

Kursvorbereitung durch Lehrende

1. `/teacher/` öffnen.
2. Kurs über New course anlegen oder vorhandenen Kurs öffnen.
3. In Overview Kursdaten prüfen.
4. In Content Textbook anlegen oder Skript hochladen.
5. Seiten prüfen, bearbeiten und speichern.
6. Skills pro Seite vergeben.
7. In Students Studierende einschreiben.
8. Studierendensicht im Dashboard kontrollieren.

Lernen durch Studierende

1. `/dashboard/index.html` öffnen.
2. Kurs auswählen.
3. Über Skript Inhalte lesen und Seiten abschließen.
4. Im Kurs-Chat Fragen stellen.
5. Über Quizzes oder Exams Übungen starten.
6. Über Games Begriffe wiederholen.
7. Punkte, Level, Streaks und Skills im Dashboard prüfen.

Quiz- und Exam-Flow

1. Das Dashboard öffnet den kursbezogenen WebSocket.
2. Das Frontend sendet `quiz_start` oder `exam_start`.
3. Der Orchestrator erzeugt Aufgaben aus RAG- oder Kurskontext.
4. Das Backend speichert `QuizAttempt` oder `ExamAttempt`.
5. Das Frontend zeigt Fragen ohne serverseitige Lösungen an.
6. Nach der Abgabe bewertet das Backend.
7. Learning/Gamification vergeben Punkte und Skill-Fortschritt.
8. Das Dashboard aktualisiert die sichtbaren Werte.

Technische Zusatzdokumentation

Neben dieser Datei existieren detaillierte technische Notizen unter `documentation/`. Diese Dateien sollten erhalten bleiben oder später geordnet in die Sphinx-Entwicklerdokumentation migriert werden.

Vorhandene Zusatzdokumente

Datei	Inhalt
<code>documentation/frontend-dashboard.md</code>	Technische Dokumentation von Dashboard-Struktur, Routes, Stores, API-Schicht, WebSocket-Protokoll und Pages.
<code>documentation/frontend-teacher.md</code>	Technische Dokumentation von Teacher-Frontend, Kursworkflow, Content-Editor, Enrollment und Stolperfallen.
<code>documentation/gamification.md</code>	Vollständige technische Beschreibung von Gamification-Modellen, Services, Signals, API und Lernaktivitätsanbindung.
<code>documentation/gamification-course-progress.md</code>	Details zu CourseProgress, Kursleveln und Kursfortschritt.
<code>documentation/gamification-api-route-fix.md</code>	Reparaturhinweis zu Gamification-API-Routen und Signalregistrierung.
<code>documentation/skills-catalog-progress-implementation.md</code>	Trennung von globalem Skill-Katalog und nutzerspezifischem SkillProgress.
<code>documentation/admin-bundle-loader-fix.md</code>	Technischer Hinweis zum Admin-Bundle-Loader.

Das Benutzerhandbuch liegt unter `docs/handbook/benutzerhandbuch.rst`. Es ist bewusst nutzerorientiert und sollte keine Branch-, Modell- oder API-Details enthalten.

Bekannte Punkte für die Übergabe

Folgenden Punkte dienen zur Orientierung, diese zeigen wo die nächste Gruppe am meisten bewirken könnte.

Performance prüfen

ELISA ist aktuell als PoC breit integriert. RAG-Suche, LLM-Aufrufe, WebSocket-Flows, Dashboard-Refreshes und Content-Synchronisation sollten mit realistischen Kursdaten getestet werden. Besonders wichtig ist, ob Quiz, Exam und Chat bei größeren Textbooks noch flüssig reagieren.

Modellwahl prüfen

Der LLM-Client nutzt aktuell Mistral mit `mistral-small-latest` und `mistral-embed`. Die nächste Gruppe sollte prüfen, ob dieses Modell für Antwortqualität, Quiz-Erzeugung, Freitextbewertung, Kosten und Latenz passt oder ob andere Modelle geeigneter sind.

Memory und Spiele schärfen

Memory, Flashcards und Hangman extrahieren Begriffe automatisch aus Kursinhalten. Das funktioniert für gut strukturierte Seiten besser als für unklare oder sehr kurze Texte. Die nächste Gruppe sollte die Begriffsauswahl fachlich verbessern, damit nicht zufällige oder wenig relevante Wörter in Spielen landen.

Sokratischen Hilfemodus ergänzen

Ein sokratischer Hilfemodus ist fachlich sinnvoll, aber im aktuellen Stand nicht als eigener Bedienmodus umgesetzt. Die nächste Gruppe kann einen Modus ergänzen, in dem ELISA nicht sofort Lösungen gibt, sondern über Rückfragen, Hinweise und kleine Zwischenschritte zum Verständnis führt.

Quiz- und Exam-Regeln entscheiden

Quiz und Exam laufen technisch über den kursbezogenen Assistant-WebSocket. Das ist für den PoC ausreichend. Für eine langfristige Architektur sollte entschieden werden, ob Quiz/Exam in diesem WebSocket bleiben oder ob ein separater API-Bereich mit klareren Tests, Historie und Konfiguration entstehen soll.

Gamification didaktisch finalisieren

Punkte, Level, Streaks, Skills und Anti-Farming-Regeln sind technisch vorhanden. Die pädagogischen Regeln sind aber noch nicht final. Die nächste Gruppe sollte festlegen, welche Aktivitäten belohnt werden, wie stark sie gewichtet sind und welche Grenzen für formale Leistungsbewertung gelten.

Lernstandsmodell erweitern

Das aktuelle Lernstandsmodell speichert bereits Kurs, letzte Seite, abgeschlossene Seiten, Kursabschluss und Quiz-Ergebnisse. Für einen ausgebauten Lernassistenten kann es fachlich erweitert werden. Als Orientierung dient das Konzept „Lernermodell für (KI-)Lernassistent“ vom 01.06.2026: Dort sind Lernplan, geplante Lerneinheiten, geplante Lernaktivitäten, geplante Lernziele, Lernprotokoll mit Chatverlauf, kursbezogenes und kursübergreifendes Chat-Gedächtnis, Reflexionsergebnisse sowie Bewertungsraster beschrieben. Die nächste Gruppe sollte prüfen, welche Teile davon in OpenBook

übernommen werden sollen und wie sie zu bestehenden Modellen wie `LearningState`, `QuizResult`, `CourseProgress` und den Assistant-Chat-Sessions passen.

Teacher-Workflow stabilisieren

Das Teacher-Frontend deckt Kursanlage, Kursdetails, Content-Editor, Upload, Skill-Tagging und Einschreibung ab. Es sollte mit echten Lehrendenszenarien geprüft werden: Kurs aus neuem Skript anlegen, Kapitel korrigieren, Skills vergeben, Studierende einschreiben und anschließend Dashboard/Quiz testen.

Testdaten und Demo-Stand vorbereiten

Viele sichtbare Funktionen brauchen passende Daten. Ohne Kurs, Library Group, Textbooks, Seiten, Skills, Enrollment und Assistant-Dokumente bleibt die Oberfläche teilweise leer. Für die Bewertung und Weiterarbeit sollte ein reproduzierbarer Demo-Stand mit Testdaten vorbereitet werden.

Abhängigkeiten und lokale Robustheit prüfen

`sqlite-vec` ist für die RAG-Integration wichtig. Die nächste Gruppe sollte prüfen, ob die Abhängigkeit reproduzierbar installiert wird und ob frische Setups ohne manuelle Nacharbeit funktionieren. Allgemeine Setup-Schritte sollten nicht hier dupliziert, sondern in der OpenBook-Installationsdokumentation gepflegt werden.

Alte Kommentare und technische Notizen bereinigen

Einige Kommentare und Zusatzdokumente stammen aus Zwischenständen. Beispiel: `src/openbook/ai/consumers/chat.py` enthält noch einen alten Hinweis auf `/ws/ai/chat`, während das Dashboard kursbezogene Kanäle verwendet. Solche Hinweise sollten bereinigt werden, damit die Doku nicht gegen den aktuellen Code arbeitet.

Empfohlene erste Schritte für die nächste Gruppe

Aktuellen integrierten Stand lokal starten und Testdaten laden oder erstellen:

1. Mit einem Teacher-Account einen Kurs im Teacher-Frontend anlegen.
2. Ein Skript hochladen, Seiten prüfen, Skills vergeben und Studierende einschreiben.
3. Mit einem Student-Account Dashboard, Skript, Chat, Quiz, Exam und Games testen.
4. Prüfen, ob Assistant-Dokumente indexiert werden und ob RAG-Kontext genutzt wird.
5. Performance und Modellqualität mit realistischen Kursseiten beobachten.
6. Didaktische Entscheidungen für Gamification, Quiz, Exam und Hilfemodus treffen.
7. Technische Detaildokumente aus `documentation/` nach Bedarf in `docs/` migrieren.
8. Benutzerhandbuch aktualisieren, sobald sich sichtbare Bedienwege ändern.

Kurzfazit

ELISA erweitert OpenBook im aktuellen Stand um mehrere zusammenhängende Lern- und KI-Bausteine. Entscheidend ist dabei weniger das einzelne Feature als die integrierte Grundlage aus Assistant/RAG, Learning, Gamification, Dashboard und Teacher-Frontend.